

Arrays, Linked Lists, and Binary Search Trees

Arrays

An array is a **fixed size**, **homogeneous** linear data structure. All elements are the same type and stored contiguously in memory.

```
std::array<int, 5> a{};
```

0	0	0	0	0
0	1	2	3	4

Accessing and Setting Elements

```
a[2];           // read – no bounds check
```

Q: What if we access `a[10]`?

```
a.at(2);        // read – aborts if out of bounds
a[2] = 99;       // write
```

find — $O(n)$

Linear scan from index 0 to the end.

```
for (size_t i = 0; i < a.size(); ++i) {
    if (a[i] == value) return true;
}
return false;
```

Big-O Notation

Big-O is the **asymptotic upper bound of a function's growth rate**.

Asymptotic means we care about behavior as n approaches infinity — we drop constants and lower-order terms. $O(2n)$ and $O(n + 100)$ are both $O(n)$.

`find` above is $O(n)$ — in the worst case we check every element.

Consider bubble sort, which is $O(n^2)$ — for every element, we scan the rest:

```
for (size_t i = 0; i < n; ++i) {
    for (size_t j = 0; j < n - i - 1; ++j) {
        if (a[j] > a[j + 1]) {
            std::swap(a[j], a[j + 1]);
        }
    }
}
```

Q: What is the best case for `find`? Does Big-O capture that?

Motivation: Arrays Are Fixed Size

Arrays require a capacity set at creation time. Too small and you run out of space. Too large and you waste memory. You must know the size upfront.

What if you don't?

That means we need a datastructure that can grow or shrink as needed. One solution is a dynamic array (`std::vector` in c++), but instead, let's look at a datastructure that allocates and removes elements as needed.

Linked Lists

What is a Linked List?

A chain of **nodes**. Each node holds a value and a pointer to the next node. No contiguous memory. No fixed size.

```
head
|
[10] -> [20] -> [30] -> nullptr
```

```
struct Node {
    int data;
    Node* next;
};
```

Q: How does a linked list differ from an array in memory?

insert_front

Linked list — $O(1)$: Create node, point it at head, update head.

```
before: head -> [10] -> [20] -> nullptr
after:  head -> [99] -> [10] -> [20] -> nullptr
```

```
void insert_front(int value) {
    head = new Node{value, head};
}
```

Array — $O(n)$: Must shift every element right first to make room.

insert 99 at front of array:

```
before: [10, 20, 30, 40, __]  size=4
```

```
step 1: [10, 20, 30, 40, 40]  shift a[3] → a[4]
step 2: [10, 20, 30, 30, 40]  shift a[2] → a[3]
step 3: [10, 20, 20, 30, 40]  shift a[1] → a[2]
step 4: [10, 10, 20, 30, 40]  shift a[0] → a[1]
step 5: [99, 10, 20, 30, 40]  write 99 at a[0]
```

Linked list touches 1 node. Array touches every element.

Q: Why is `insert_front` $O(1)$ for linked lists but $O(n)$ for arrays?

insert_back

Linked list — $O(n)$: Walk to the last node, attach new node.

```
void insert_back(int value) {
    Node* node = new Node{value, nullptr};
    if (head == nullptr) { head = node; return; }
    Node* curr = head;
    while (curr->next != nullptr) { curr = curr->next; }
    curr->next = node;
}
```

Array — $O(1)$: Write at `data[size]`, increment `size`.

Q: Why does the linked list lose the advantage here?

remove_front

Linked list — $O(1)$: Advance `head`, delete old node.

Array — $O(n)$: Must shift everything left.

remove_back

Linked list — $O(n)$: Walk to second-to-last node, delete last, set `next = nullptr`.

Array — $O(1)$: Decrement `size`.

Q: Do we need to erase the value at `data[size]`? Why or why not?

find — $O(n)$

Walk forward until value found or end reached. Same as array.

Linked List with Tail

Add a `tail` pointer to track the last node.

```
head          tail
|             |
[10] -> [20] -> [30] -> [40] -> nullptr
```

`insert_back` becomes $O(1)$.

Q: Does a tail pointer make `remove_back` $O(1)$? Why or why not?

Doubly Linked List

Each node has both `next` and `prev` pointers.

```
nullptr <- [10] <-> [20] <-> [30] -> nullptr
```

Q: What operations become easier with a `prev` pointer? What gets harder?

Q: If a doubly linked list has more information, why is `find` not faster?

Complexity Summary — Linked Lists

Operation	Array	Linked List	LL + Tail	Doubly LL
<code>insert_front</code>	$O(n)$	$O(1)$	$O(1)$	$O(1)$
<code>insert_back</code>	$O(1)$	$O(n)$	$O(1)$	$O(n)$
<code>remove_front</code>	$O(n)$	$O(1)$	$O(1)$	$O(1)$
<code>remove_back</code>	$O(1)$	$O(n)$	$O(n)$	$O(n)$
<code>find</code>	$O(n)$	$O(n)$	$O(n)$	$O(n)$

Q: We've improved some operations with linked lists. What did we give up?

Binary Search Trees

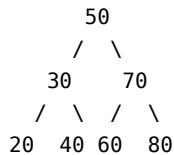
`find` is $O(n)$ for every structure so far. Can we do better?

What is a BST?

A tree where each node has at most two children. For every node:

- Left subtree: all values **less than** the node
- Right subtree: all values **greater than** the node

```
struct BSTNode {
    int data;
    BSTNode* left;
    BSTNode* right;
};
```



Example tree above is produced by inserting: 50, 30, 70, 20, 40, 60, 80

insert — $O(\log n)$ average

Compare to current node, recurse left or right until an empty spot is found.

```
BSTNode* insert(BSTNode* node, int value)
{
    if (node == nullptr)
    {
        return new BSTNode{value, nullptr, nullptr};
    }
    if (value < node->data)
    {
        node->left = insert(node->left, value);
    }
    else if (value > node->data)
    {
        node->right = insert(node->right, value);
    }
    return node;
}
```

Q: What is the time complexity?

Our simple for-loop approach doesn't work anymore since we are using recursion.

A balanced BST adds one level for every doubling of nodes:

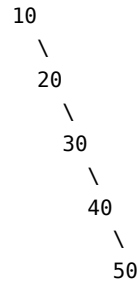
```
1 node → height 0 (just root)
3 nodes → height 1
7 nodes → height 2
15 nodes → height 3
```

- Each level holds twice as many nodes as the previous. A tree of height h holds at most $2^{h+1} - 1$ nodes.
- Flip it around: given n nodes, height h satisfies $2^h \sim n$, so $h \sim \log_2(n)$.
- Insert follows exactly one path from root to leaf — at most $h = \log_2(n)$ steps.

This assumes the tree is balanced, like our example above. **Q:** How could we build an unbalanced tree?

Insert in sorted order degenerates to a linked list.

insert 10, 20, 30, 40, 50:



Q: How would you avoid this? (leads to balanced BSTs — AVL, Red-Black)

find — $O(\log n)$ average

Same traversal as insert.

Q: Why is find $O(\log n)$ in a BST but $O(n)$ in a linked list?

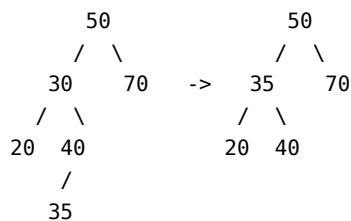
remove — $O(\log n)$ average

Three cases:

1. **No children** — delete the node.
2. **One child** — replace node with its child.
3. **Two children** — replace with in-order successor, delete successor.

The **in-order successor** of a node is the next value in sorted order — the smallest value still larger than the node. Found by going right once, then left as far as possible. It has at most one child (no left child by definition), so deleting it reduces to case 1 or 2.

remove 30 (two children):



```

BSTNode* remove(BSTNode* node, int value)
{
    if (node == nullptr) return nullptr;

    if (value < node->data)
    {
        node->left = remove(node->left, value);
    }
    else if (value > node->data)
    {
        node->right = remove(node->right, value);
    }
    else
    {
        if (node->left == nullptr)
        {

```

```

        BSTNode* tmp = node->right;
        delete node;
        return tmp;
    }
    if (node->right == nullptr)
    {
        BSTNode* tmp = node->left;
        delete node;
        return tmp;
    }

    // two children: replace with in-order successor
    BSTNode* successor = node->right;
    while (successor->left != nullptr) { successor = successor->left; }
    node->data = successor->data;
    node->right = remove(node->right, successor->data);
}
return node;
}

```

Complexity Summary — BST

Operation	Average	Worst
insert	$O(\log n)$	$O(n)$
find	$O(\log n)$	$O(n)$
remove	$O(\log n)$	$O(n)$

Sorted Array

BSTs give $O(\log n)$ find — but what if we just kept an array sorted?

find — $O(\log n)$

Binary search: compare middle element, recurse left or right.

insert — $O(n)$

Binary search finds the position in $O(\log n)$. Shifting to make room costs $O(n)$. Shift dominates.

Q: If find is $O(\log n)$, why is insert still $O(n)$?

Q: When would you prefer a sorted array over a BST?

Complexity Summary — Sorted Array vs BST

Operation	Sorted Array	BST
insert	$O(n)$	$O(\log n)$
find	$O(\log n)$	$O(\log n)$
remove	$O(n)$	$O(\log n)$

Array with size and capacity

Now that we understand insert and remove, we can apply these operations back to arrays by tracking:

- **capacity** — max number of elements the array can hold
- **size** — number of elements currently stored

capacity = 5, size = 3

10	20	30		
0	1	2	3	4
^size				

Overall Comparison

Operation	Array	Linked List	BST	Sorted Array
insert_front	$O(n)$	$O(1)$	—	$O(n)$
insert_back	$O(1)$	$O(n)$	—	$O(n)$
remove_front	$O(n)$	$O(1)$	—	$O(n)$
remove_back	$O(1)$	$O(n)$	—	$O(1)$
find	$O(n)$	$O(n)$	$O(\log n)$	$O(\log n)$

Q: Given this table, when would you reach for each data structure?